MedEx: Full-Stack Intern Take-Home Exercise

Coupon & Checkout Calculator

Thank you for your interest in an internship at MedEx.

MedEx is a healthcare marketplace. Customers in Thailand, Nepal and India use it to book health check-ups, lab tests and wellness treatments. This exercise is a small, self-contained version of a real part of our checkout. It needs a **backend** and a **frontend**. The logic is the hard part, so get that right first.

How this works

- **Time:** about **8–10 hours** in total. You have **7 days** from the day you receive this. If you run out of time, stop and write in your README what is missing. An honest README counts in your favour.
- **Backend:** Node.js, ideally with TypeScript. Express, Fastify or NestJS are all fine (we use NestJS, so that's a small plus). **Coupon usage must survive a server restart**, so use SQLite, a JSON file or any database you like.
- **Frontend:** React, with Vite or Next.js. Any UI library is fine.
- **Data:** `data/checkout.json` (fictional) holds services, customers and coupons.
- **Examples:** the data file contains `examples`, each with a request and the **exact** response we expect. Write automated tests that run every example and compare the results.
- **AI tools are allowed.** If you use them, say where in your README. In the interview we will ask you to explain your code and change it live, so make sure you understand every line.

What to send us

1. A link to a Git repository with `backend/` and `frontend/` folders and a commit history we can follow.
2. A `README.md` covering:
    - how to run both parts,
    - how your calculation works, and how you avoided money rounding errors,
    - what you would improve with more time.
3. Tests, at least for the calculation. They must include every example in the data file.

How we evaluate

Area	What we look for
Logic (most important)	Correct results to the last satang, including the tricky cases.
API	Clear endpoints, input validation, sensible errors and status codes.
Frontend	Easy to use, and handles loading and error states.
Code quality	Readable, sensibly organised, with tests.
Communication	Your README explains your thinking.

The problem

> *"I had two coupon codes. One worked and one didn't, and I couldn't tell why. And the total didn't match what I worked out myself."*

Build the checkout calculator. A customer has a cart of services and enters up to two coupon codes. Your backend works out:

- which codes apply, and why the others don't;
- how much each one takes off, and from which items;
- the VAT and the final total.

The frontend shows all of this clearly.

Money

- All amounts are Thai baht, sent and returned as **strings with 2 decimals**, for example `"3450.50"`.
- **Floating-point maths is not accurate enough for money:** in JavaScript, `0.1 + 0.2 !== 0.3`. Choose a safe approach and explain it.
- "Round" always means **round half up to 2 decimals**, so `0.125` becomes `0.13`.

Rules

1. Lines. Each line's amount is `price × quantity`. The subtotal is the sum of all line amounts.

2. Codes.

- **Format:** codes ignore case and surrounding spaces, so `" lab15 "` is `LAB15`.
- **Repeats:** a code entered twice is rejected the second time with *"This code is already applied."*.
- **Limit:** at most **2** different codes per order. Any further code is rejected with *"You can use up to 2 codes per order."*.

3. Order of evaluation.

- Codes that are **not** `stackable` are evaluated first, then stackable ones. Within each, follow the order the customer entered them.
- Results are reported in the **order the customer entered them**.

4. Checks. Each code is checked in this order. The first check that fails gives the reason:

#	Check	Message
a	Code exists	*We don't recognise this code.*
b	`status` is `active`	*This code is no longer active.*
c	Today is on or after `startsOn`	*This code starts on {startsOn}.*
d	Today is on or before `endsOn`	*This code expired on {endsOn}.*
e	`usedCount < usageLimit` (when there is a limit)	*This code has been fully used.*
f	Customer is logged in, if the code has `perCustomerLimit` or `firstOrderOnly`	*Log in to use this code.*
g	This customer has used it fewer than `perCustomerLimit` times	*You've already used this code.*
h	`firstOrderOnly`: the customer has no previous orders	*This code is only for your first order.*
i	A non-stackable code can't join another non-stackable code already applied	*This code can't be combined with {CODE}.*
j	At least one cart line is in the code's `scope`	*No items in your cart qualify for this code.*
k	The eligible amount is at least `minSpend`	*Spend ฿{missing} more on eligible items to use this code.*

"Today" means the calendar date in **Asia/Bangkok (UTC+07:00)**, and `endsOn` is included. In the `k` message, `{missing}` uses a thousands separator and 2 decimals, for example `฿1,234.50`.

5. Eligible amount.

- The eligible lines are the lines in the coupon's `scope`: all lines, one service `type`, or one `providerId`.
- The eligible amount is the sum of those lines **after any discounts from coupons evaluated earlier**.

6. Discount.

- **Percent:** round(eligible amount × value / 100). If `maxDiscount` is set, the discount is no more than that.
- **Fixed:** the coupon value, but never more than the eligible amount.

7. Splitting the discount across lines. We need to know how much discount each line received, for refunds.

1. Give each eligible line round(discount × line amount / eligible amount), using the line's amount after earlier discounts.
2. Because of rounding, the shares may not add up to the discount exactly. Add the difference (positive or negative) to the eligible line with the largest amount. If two lines tie, use the first one.

8. Totals.

- net amount = subtotal − all discounts
- VAT = round(net amount × 7 / 100)
- total = net amount + VAT

API

```
GET  /api/services
POST /api/quote    { "lines": [{ "serviceId": "S1", "quantity": 1 }], "couponCodes": ["WELCOME10"],
                     "customerId": "C1" | null, "at"?: "2026-11-02T10:00:00+07:00" }
POST /api/orders   (same body as /api/quote)
```

`at` is optional. It lets tests fix the current time; without it, use the real time.

`POST /api/quote` returns exactly this shape. The data file has full examples.

```
{
  "lines": [ { "serviceId": "S1", "quantity": 1, "amount": "1200.00",
               "discounts": { "WELCOME10": "77.41" }, "amountAfterDiscounts": "1122.59" }, … ],
  "coupons": [ { "code": "WELCOME10", "status": "applied", "discount": "300.00" },
               { "code": "FLASH25", "status": "rejected", "reason": "This code expired on 2026-10-07." } ],
  "subtotal": "4650.50", "discountTotal": "300.00", "netAmount": "4350.50", "vat": "304.54", "total":
"4655.04"
}
```

Validation:

- An unknown `serviceId` returns `400`.
- `quantity` must be a whole number from 1 to 20.

Placing an order (`POST /api/orders`)

1. It calculates the quote again.
2. It records the order and increases each applied coupon's `usedCount`, and the customer's count in `usedBy`. The customer's `ordersCount` also goes up, so `firstOrderOnly` codes stop working for them.
3. These updates must be **safe when two orders arrive at the same moment**. `LASTONE` has exactly one use left: if two customers order with it at the same time, only one gets it.
4. If a code that applied in the customer's last quote no longer applies, don't place the order. Return `409` with the new quote so the customer can confirm the new total.

Frontend

1. **Cart:** add services from the list, change quantities and remove lines.
2. **Coupons:** a code field; pressing Enter applies the code. Applied codes show as chips with a remove button.
3. **Clear feedback:** every rejected code shows its reason in plain words, next to that code.

4. **Breakdown:** each line shows its amount, its discount from each code, and its amount after discounts. Then show subtotal, discounts, VAT and total. Everything updates when the cart or the codes change.
5. **Placing the order:** on success, show a confirmation. On `409`, show exactly what changed (for example, "*LASTONE is no longer available; your total is now ฿1,284.00*") and ask the customer to confirm.
6. **States:** loading and errors, and it works at 375 px wide.

Questions?

If something is unclear, make a sensible decision, write it down in your README and carry on. If you're blocked, email us and we'll reply within one working day.

Good luck!